

LZW Compression Tool

(a) What I Built

For this project I implemented the Lempel–Ziv–Welch (LZW) compression algorithm in C++ and built a command-line tool for compressing and decompressing files.

LZW is a dictionary-based lossless compression algorithm that replaces repeated sequences with integer codes [1]. My implementation begins with the 256 possible byte values, with new dictionary entries starting at code 256 [2].

The project is split into three main components:

- `lzw.cpp` — compression and decompression
- `bit_io.cpp` — variable-width bit packing
- `file_io.cpp` — binary file handling and the custom compressed format

My first working format, LZW₁, stored every generated code using a fixed 16-bit representation. This made the initial implementation simple to verify, but it was inefficient. A 551-byte sample became 592 bytes after compression because every code occupied 16 bits even while the dictionary was still small.

I therefore created LZW₂, which uses variable-width codes. Codes begin at 9 bits and increase to 10, 11 and eventually 16 bits as the dictionary grows. Increasing code widths as an LZW dictionary expands is also used in documented LZW-based formats such as GIF [3].

This required a Bit Writer and Bit Reader so codes such as 9-bit and 10-bit values could be packed efficiently across normal 8-bit byte boundaries.

I also improved how files are processed in memory. The initial implementation loaded the entire input file into a `std::string` and stored the complete sequence of generated codes in a `vector<int>`.

The final implementation instead processes files as streams. Input bytes are read progressively, and completed LZW codes are sent directly to the Bit Writer. During decompression, reconstructed sequences are written directly to the output stream.

This reduces memory usage while keeping one LZW dictionary active for the entire file.

(b) Track B – Implementation

The final program is a command-line compression tool with two main operations:

```
./lzw compress <input> <output>
```

and:

```
./lzw decompress <input> <output>
```

During compression, the program:

1. Opens the input file as a binary stream.
2. Processes bytes using the LZW dictionary.
3. Generates dictionary codes.
4. Packs those codes using variable-width bit encoding.
5. Writes them directly to an LZW2 file.

The compressed file begins with a four-byte LZW2 header so that the decompressor can confirm that the file uses the expected format.

Decompression reverses this process. The Bit Reader reconstructs each code from the packed bit stream while the decoder rebuilds its dictionary and writes the decoded data directly to the output file.

The CLI also reports:

- original file size
- compressed file size
- number of generated LZW codes
- bytes saved
- compression ratio
- percentage size reduction
- execution time

It also warns the user when compression produces a file larger than the original.

Worked Example

For a normal prose sample:

```
Original size: 6833 bytes  
Compressed size: 3285 bytes  
LZW codes: 2422  
Bytes saved: 3548 bytes  
Compression ratio: 2.08:1  
Size reduction: 51.92%
```

The compressed file was then decompressed and compared with the original using `cmp`. No differences were reported, confirming that the compression was lossless.

Compression Results

Input	File Type	Original	Compressed	Ratio	Reduction
Highly repetitive text	Text	415000 B	12558 B	33.05:1	96.97%
Normal prose	Text	6833 B	3285 B	2.08:1	51.92%
C++ source code	Source code	1770 B	943 B	1.88:1	46.72%
Random binary data	Binary	102400 B	138634 B	0.74:1	-35.38%
data.csv	Structured CSV	366842 B	81852 B	4.48:1	77.69%
image.bmp	Uncompressed BMP image	786486 B	13209 B	59.54:1	98.32%
tone.wav	Uncompressed WAV audio	441044 B	112056 B	3.94:1	74.59%
document.pdf	PDF document	743 B	497 B	1.49:1	33.11%
archive.zip	ZIP archive	3155 B	4143 B	0.76:1	-31.32%

The results show that compression effectiveness depends heavily on the structure of the input.

The generated BMP image achieved the highest reduction at 98.32% because its repeated stripe pattern contained large amounts of repeated pixel data. Similarly, the structured CSV and generated WAV tone compressed well because both contained repeated patterns that could be represented efficiently by the LZW dictionary.

The ZIP archive became 31.32% larger because its contents had already been compressed, leaving relatively little repeated structure for LZW to exploit. Random binary data behaved similarly and increased by 35.38%.

The PDF sample achieved a smaller 33.11% reduction. However, this file was only 743 bytes, so fixed format and header overhead have a much larger effect than they would for a larger document.

These experiments reinforced that the file extension itself does not determine how well LZW performs. The amount and structure of repeated byte sequences in the underlying data are more important.

(c) What I Learned

The biggest thing that surprised me was that implementing LZW correctly did not automatically mean that I had built an effective compressor.

My first LZW₁ implementation compressed and decompressed files correctly, but a 551-byte sample became 592 bytes. The problem was not the LZW algorithm itself, but how I represented its output. Using 16 bits for every code wasted space while the dictionary was still small.

This showed me that compression performance depends both on the compression algorithm and on how the encoded data is represented. Moving to variable-width codes substantially reduced this overhead.

I also learned how strongly LZW depends on the structure of the input. Highly repetitive data achieved a 96.97% reduction, while random binary data became 35.38% larger. I expected random data to compress poorly, but the amount of expansion was larger than I initially expected.

Another important concept was synchronisation between compression and decompression. The compressor does not store the complete dictionary in the file. Instead, the decompressor reconstructs compatible dictionary entries while reading the code stream.

This means dictionary growth and code-width transitions must occur at the correct points on both sides. If the writer begins producing 10-bit codes while the reader still expects 9-bit codes, the remaining bit stream becomes misaligned and the decoded data becomes incorrect.

Variable-width bit packing was therefore one of the most technically difficult parts of the project. Because values such as 9-bit and 10-bit codes do not align to 8-bit file bytes, I had to understand how to buffer, shift and extract individual bits across byte boundaries.

I also improved the scalability of the program. My first implementation loaded the complete input and generated code vector into memory. The final implementation operates on streams and writes data incrementally.

This means memory usage is mainly determined by the bounded LZW dictionary and small buffering state rather than the complete file size. It was important to keep one dictionary active across the whole file, because independently compressing chunks would lose patterns that cross chunk boundaries.

Testing also exposed cases that were not obvious during normal use. One GoogleTest revealed that the decoder did not explicitly validate the first code. Since the dictionary initially only contains codes 0–255, the first code must belong to that range. Adding this validation fixed the failing test.

Overall, the project progressed through three main stages:

1. implementing a correct LZW algorithm
 2. improving storage efficiency with variable-width codes
 3. improving scalability through streaming file processing
-

(d) AI Use

I used ChatGPT regularly throughout the project as a learning, and debugging aid.

Tools and Amount of Use

ChatGPT was the main generative AI tool I used. I used it throughout most stages of the project, particularly while learning LZW, debugging C++ code, and configuring tests.

What I Used AI For

I used generative AI for:

- understanding the LZW algorithm
- finding material for further reading
- generating different sample inputs for testing
- debugging compiler and linker errors
- constructing C++ compilation commands
- configuring Google Test on macOS
- explaining binary streams and bit manipulation

Example 1 – Inefficient Fixed 16-bit Representation

The first file-format approach that was suggested by AI and one that I used stored every LZW code using a fixed 16-bit value.

This was useful as an initial implementation because it made the encoded output simple to write and read, but it was not an efficient representation of the compressed data.

The problem became clear during testing when a 551-byte input produced a 592-byte compressed file. The LZW algorithm itself was working correctly, but the storage format introduced too much overhead because even small dictionary codes were always written using the full 16 bits.

I improved this by replacing the fixed-width LZW₁ representation with LZW₂, which begins with 9-bit codes and increases the width only as the dictionary grows.

This was an important example of why an implementation can be logically correct while still producing poor practical results, and why generated suggestions need to be evaluated beyond whether they simply work.

Example 2 – Incorrect Compression Statistics

Another AI suggestion incorrectly reused the compression ratio when calculating percentage size reduction.

For a highly repetitive test file this produced an impossible reduction value of more than 3000%.

I manually checked the calculation and corrected the formula to:

$1 - (\text{compressed size} / \text{original size})$

The correct reduction for the test was 96.97%.

This demonstrated the importance of sanity-checking numerical output instead of assuming it is correct because the program successfully runs.

What I Understood vs What I Initially Took on Trust

By the end of the project, I was confident in my understanding of:

- the core LZW compression algorithm
- dictionary growth and reconstruction
- the special LZW decompression case
- variable-width code encoding
- bit packing across byte boundaries
- writer and reader synchronisation
- how input structure affects compression
- why streaming improves memory scalability

I initially relied more heavily on AI guidance for lower-level C++ details such as binary streams, bit manipulation and Google Test build configuration.

I verified these suggestions using compiler feedback, unit tests, round-trip file comparisons and further reading rather than accepting generated code without testing it.

Overall, generative AI accelerated the learning and debugging process, but its suggestions still required verification and independent understanding.

References

[1] T. A. Welch, "A Technique for High-Performance Data Compression," *Computer*, vol. 17, no. 6, pp. 8–19, 1984.

<https://doi.org/10.1109/MC.1984.1659158>

[2] IBM, "Compression processing," IBM Documentation.

<https://www.ibm.com/docs/en/zos/2.5.0?topic=csrcmpsc-compression-processing>

[3] CompuServe, "Graphics Interchange Format Version 89a," Appendix F: Variable-Length-Code LZW Compression.

<https://giflib.sourceforge.net/gifstandard/GIF89a.html>

